

Author

Name: Prajin GN

Roll Number: 24f2009067

Email: 24f2009067@ds.study.iitm.ac.in

About Me:

I am a Computer Science student with a strong interest in backend development and full-stack web technologies. I have experience working with Vuejs, Flask, SQLAlchemy, and JavaScript, and I enjoy building structured, scalable web applications while improving my problem-solving skills.

AI/LLM usage

I used **ChatGPT (GPT-5)** to assist in generating mock data and to get UI ideas. The extent of AI/LLM usage is around **15–20%**, limited to suggestions on UI and layout. All final Implementation logic, debugging, and integration were done manually.

Description

Project Title: Placement Portal Application

Problem Statement:

Placement Portal Application V2 is a web application that digitizes the campus recruitment process by providing separate portals for Admin, Companies, and Students. The system manages company approvals, placement drives, student applications, interview scheduling, application tracking, asynchronous report generation, and API caching using Flask, Vue 3, Celery, and Redis.

Approach:

The app is built using Vuejs 3 for its frontend and Flask for its backend, it has responsive UI build using bootstrap. Allows role-based access to Admin, Students and Companies allowing the creation and management of drives, student applications and Interviews. The admin controls access to students and companies. Async jobs run on the backend to generate Reports and send email reminders to students.

Technologies used

Technology / Library	Purpose
Flask	Backend REST API
Flask-SQLAlchemy	ORM and database operations

Flask-JWT-Extended

Authentication using JWT

Flask-Caching

Redis-based API caching

Flask-CORS

Cross-Origin Resource Sharing

SQLite

Relational database

Vue 3

Frontend framework

Vue Router

Client-side routing

Bootstrap 5

Responsive UI

Bootstrap Icons

Icons

Redis

API cache, Celery broker/ results backend

Celery

Background jobs

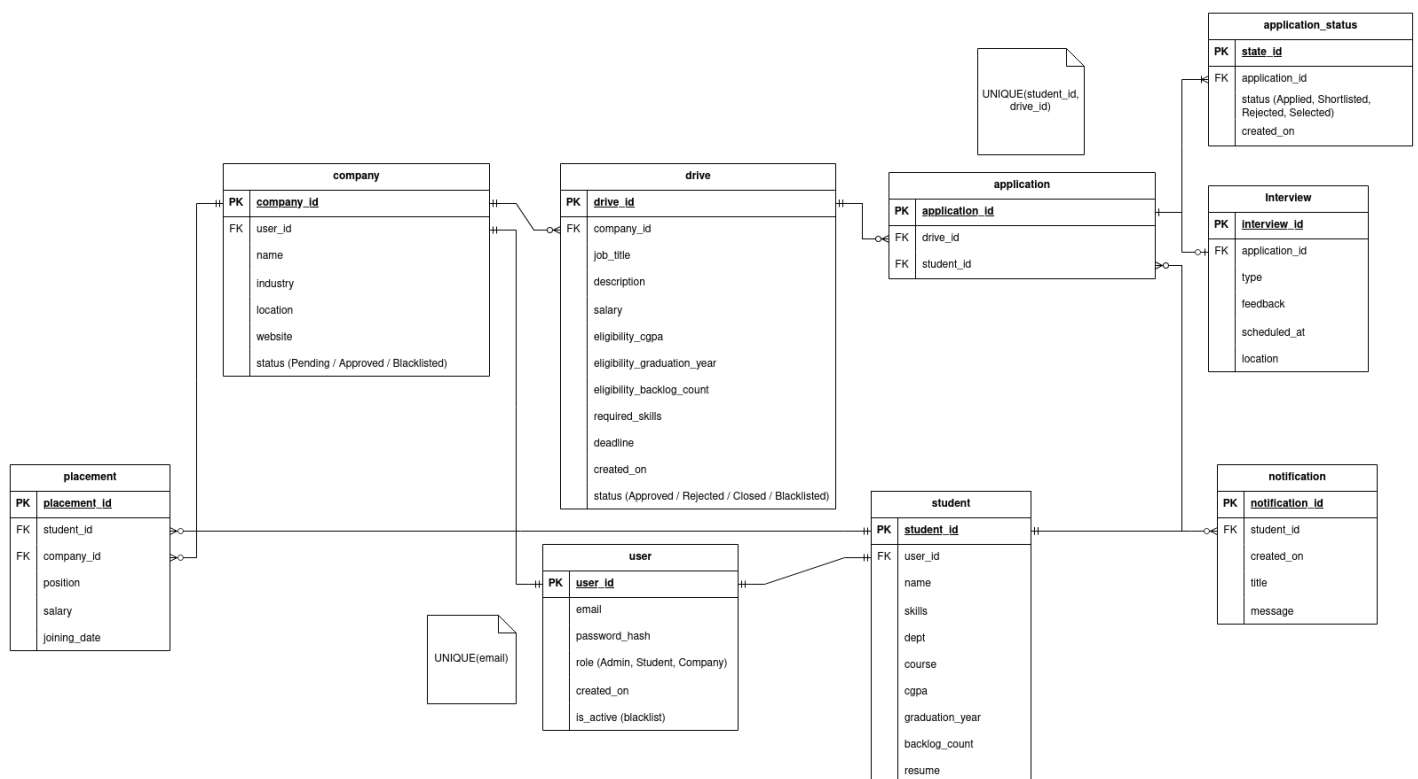
Jinja2

Admin monthly report (html file generation)

Vite

Frontend build tool

DB Schema Design



Tables

- **User** — Stores user account credentials and role information (user_id, email, password_hash, role, created_on, is_active).
- **Student** — Stores student profile, academic details, and resume information (student_id, user_id, name, skills, dept, course, cgpa, graduation_year, backlog_count, resume).
- **Company** — Stores registered company profile and approval status (company_id, user_id, name, industry, location, website, status).
- **Drive** — Stores placement drive details, eligibility criteria, and recruitment status (drive_id, company_id, job_title, description, salary, eligibility_cgpa, eligibility_graduation_year, eligibility_backlog_count, required_skills, deadline, status).
- **Application** — Stores student applications submitted for placement drives (application_id, drive_id, student_id).
- **Application_Status** — Stores the complete status history of each application (state_id, application_id, status, created_on).
- **Interview** — Stores interview schedules, type, location, and feedback for applications (interview_id, application_id, type, feedback, scheduled_at, location).
- **Notification** — Stores notifications sent to students (notification_id, student_id, created_on, title, message).
- **Placement** — Stores final placement details of selected students (placement_id, student_id, company_id, position, salary, joining_date).

Relationships

- **1:1** → User ↔ Student
- **1:1** → User ↔ Company
- **1:N** → Company → Drive
- **1:N** → Student → Application
- **1:N** → Drive → Application
- **1:N** → Application → Application Status
- **1:N** → Application → Interview
- **1:N** → Student → Notification
- **1:N** → Student → Placement
- **1:N** → Company → Placement
- **M:N** → Student ↔ Drive (via Application)

Constraints

- **UNIQUE (student_id, drive_id)** — prevents duplicate applications for the same placement drive.
- **Unique email** in the **User** table ensures each account is registered only once.
- **Foreign key constraints** maintain referential integrity between related tables.
- **One-to-one relationship constraints** on user_id ensure each User is associated with at most one Student or Company profile.

API Design

- Most API endpoints require a valid JWT access token to respond. Use '/api/auth/login' to generate a JWT token.

Method	API Endpoint	Description
POST	/api/auth/users	Register a new user
GET	/api/auth/users	Get the logged-in user's info
POST	/api/auth/login	Log in with email & password, returns a JWT access token.
GET	/api/students	Get the logged-in student's profile.
POST	/api/students	Create or update the logged-in student's profile .
GET	/api/students/resume/<id>	Download a student's resume file by student id.
GET	/api/students/dashboard	Get student dashboard data
GET	/api/students/company/<user_id>	Get an approved company's public profile and its open (approved) drives.
GET	/api/students/drives/<drive_id>	Get a single drive's details
POST	/api/students/drives/<drive_id>/applications	Apply to a drive (if eligible).
GET	/api/students/notifications	Get the student's unseen notifications.
PUT	/api/students/notifications/<notification_id>/seen	Mark a notification as seen.
GET	/api/students/history	Get full application history
GET	/api/students/report	Trigger an async job to generate the student's report (celery)
GET	/api/company	Get the logged-in company's profile.
POST	/api/company	Create or update the company's profile
GET	/api/company/dashboard	Get company dashboard data
POST	/api/company/drives	Post a new drive
GET	/api/company/drives/<drive_id>	Get a drive's details and applications

PUT	/api/company/drives/<drive_id>/<action>	(close / update) drive
PUT	/api/company/applications/<application_id>/<status>	Mark an application as (select / shortlist / reject)
POST	/api/company/report	Trigger an async job to generate the company's report (celery)
GET	/api/admin/companies	List/search all companies (by status, name, email, industry).
PUT	/api/admin/companies/<user_id>/approve	Approve a pending company and activate its user account.
DELETE	/api/admin/companies/<user_id>/remove	Blacklist a company; cascades to remove its drives and mark related applications company_removed.
GET	/api/admin/students	List/search all students
DELETE	/api/admin/students/<user_id>/remove	Blacklist a student; marks their applications student_removed.
GET	/api/admin/applications	search all applications or get one by id.
GET	/api/admin/drives	List/search/filter all drives
GET	/api/admin/drives/<drive_id>	Get a single drive's full details.
PUT	/api/admin/drives/<drive_id>/approve	Approve a pending drive.
PUT	/api/admin/drives/<drive_id>/close	Close a drive.
DELETE	/api/admin/drives/<drive_id>/remove	Blacklist a drive & marks related applications drive_removed.

Architecture and Features

The project follows a client-server architecture. The frontend is implemented using Vue 3 with Vue Router and Bootstrap, while the backend is built using Flask and SQLAlchemy. REST APIs connect both layers using the Fetch API. Redis is used for caching frequently accessed endpoints, while Celery executes scheduled and asynchronous tasks such as reminder emails, monthly reports, and CSV exports. Database models, controllers, routes, utilities, and workers are organized into separate modules to improve maintainability.

Video

<https://drive.google.com/file/d/1nwv1Q-syg2SnIqpuNnV6Wb8zi58aTBJD/view?usp=sharing>